

MEMORIA TÉCNICA: RECONOCIMIENTO DE CARACTERES (OCR) MEDIANTE MLP

1. Selección y Preparación del Dataset

- **Dataset elegido:** MNIST (Dígitos manuscritos).
- **Descripción:** Un conjunto de 70,000 imágenes de dígitos del 0 al 9.
- **Características Técnicas:** Cada muestra es una imagen en escala de grises de 28 x 28 píxeles.
- **Preprocesado realizado:**
 - **Normalización:** Se han escalado los valores de los píxeles al rango [0, 1] dividiendo por 255 para mejorar la convergencia del modelo.
 - **Aplanado (Flattening):** Las imágenes se transformaron de matrices 2D a vectores de 784 dimensiones ($28 \times 28 = 784$) para ser compatibles con la entrada del Perceptrón Multicapa.

2. Metodología

Estrategia de Implementación

Se ha seguido la **Ruta A (Uso de Librerías)** utilizando el framework Scikit-Learn. Se ha empleado la clase MLPClassifier debido a su versatilidad para gestionar hiperparámetros y su integración con herramientas de optimización sistemática.

Diseño del Experimento (Grid Search)

Para evitar el ajuste manual aleatorio, se implementó GridSearchCV explorando el siguiente diccionario de hiperparámetros:

- **Capas ocultas:** [(50,), (100,)].
- **Funciones de activación:** ['relu', 'logistic'].
- **Tasa de aprendizaje (Learning Rate):** [0.001, 0.01].

3. Resultados y Optimización

Configuración Ganadora

Tras la búsqueda sistemática, el modelo óptimo seleccionado fue:

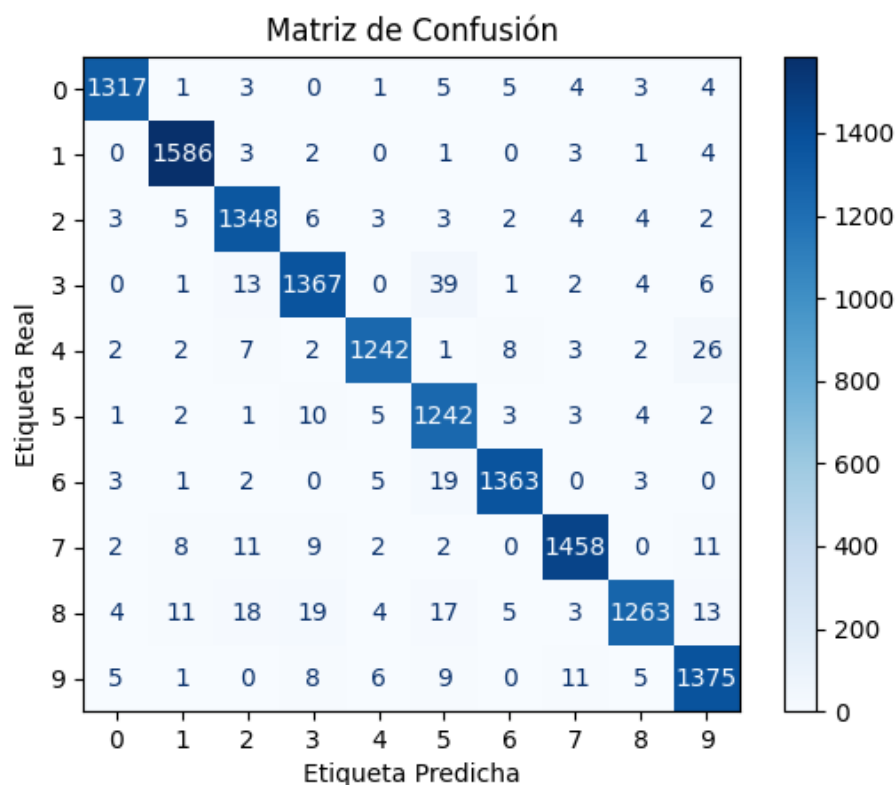
- **Función de activación:** relu.
- **Arquitectura:** 1 capa oculta con 100 neuronas.
- **Tasa de aprendizaje inicial:** 0.01.

Métricas de Evaluación

```
C:\Users\IVANA>start
C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
for configuration encountered: {'activation': 'relu', 'hidden_layer_sizes': (100,), 'learning_rate_init': 0.01}
C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\skslearn\neural_network\_multilayer_perceptron.py:785: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(C:\Users\IVANA\Desktop>
```

- **Precisión final (Accuracy): 96.86%.**

Matriz de Confusión

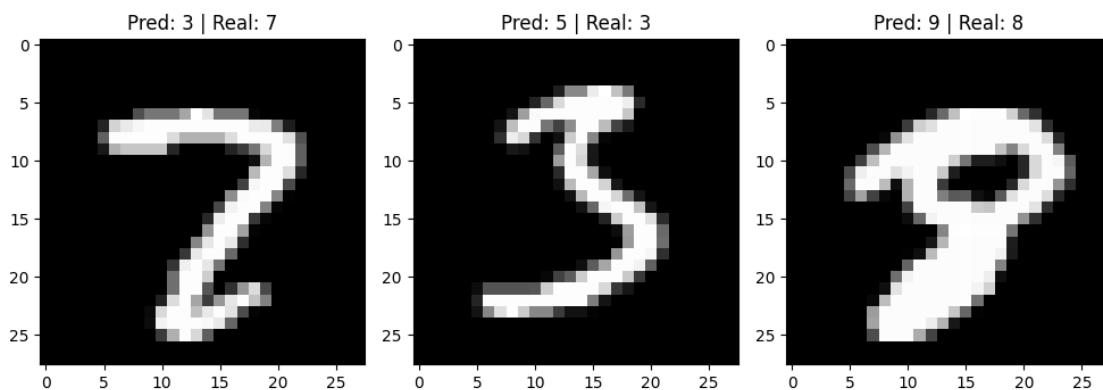


Análisis de la Matriz:

- **Aciertos:** El dígito **1** es el más preciso con 1,586 aciertos.
- **Confusiones Clave:** Se detecta una dificultad recurrente en distinguir el **3 del 5** (39 fallos) y el **4 del 9** (26 fallos), debido a la similitud morfológica de sus trazos.

4. Análisis Crítico de Errores

Se han identificado visualmente casos donde la red falló para comprender sus limitaciones:



1. **Caso 1 (Real 7, Predicho 3):** El siete presenta una curvatura excesiva en la base que la red interpreta como la parte inferior de un tres.
2. **Caso 2 (Real 3, Predicho 5):** El trazo superior del tres es muy cerrado, asemejándose al bucle característico de un cinco.
3. **Caso 3 (Real 8, Predicho 9):** La parte inferior del ocho está poco definida o "apretada", lo que induce a la red a identificarlo como un nueve con trazo grueso.

5. Conclusiones

El Perceptrón Multicapa ha demostrado una alta eficacia para el reconocimiento de caracteres básicos con una precisión cercana al 97%. El uso de Grid Search fue fundamental para descartar configuraciones menos eficientes (como la activación logística en este caso específico) de forma automática.

Como limitación técnica, se observaron avisos de convergencia (ConvergenceWarning), indicando que el modelo podría mejorar con un mayor

número de iteraciones o una arquitectura más profunda. Sin embargo, los fallos analizados demuestran que muchas de las clasificaciones erróneas se deben a la ambigüedad humana al escribir, más que a una deficiencia del algoritmo.

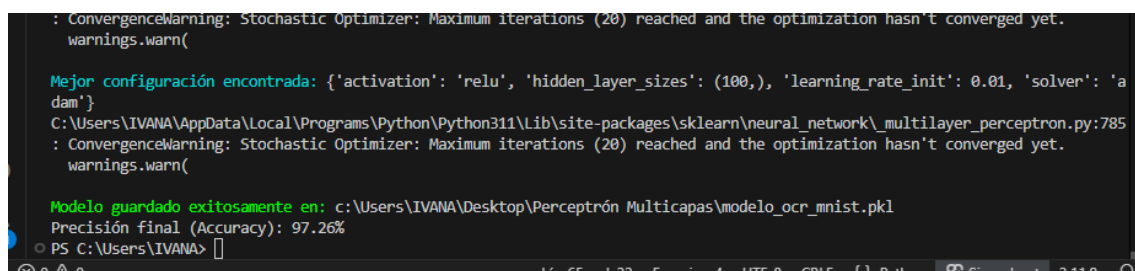
6. Ampliación y pruebas en entorno real

Como valor añadido al proyecto, se han implementado funcionalidades que permiten la transición del modelo desde un entorno de entrenamiento a un entorno de uso práctico (producción).

6.1. Persistencia del modelo

En lugar de entrenar el Perceptrón Multicapa cada vez que se requiere realizar una predicción, se ha implementado la técnica de persistencia. Se ha utilizado la librería **joblib** para exportar el modelo final entrenado (best_mlp) a un archivo binario con extensión .pkl.

- **Justificación:** joblib es significativamente más eficiente que pickle al trabajar con objetos de Scikit-Learn que contienen grandes matrices de datos (pesos de las neuronas).
- **Portabilidad:** El uso de rutas relativas mediante el módulo os de Python garantiza que el modelo pueda ser cargado en cualquier ordenador sin necesidad de re-entrenar con las 70,000 imágenes del dataset original.



```
: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(

Mejor configuración encontrada: {'activation': 'relu', 'hidden_layer_sizes': (100,), 'learning_rate_init': 0.01, 'solver': 'adam'}
C:\Users\IVANA\AppData\Local\Programs\Python\Python311\Lib\site-packages\sklearn\normal_network\_multilayer_perceptron.py:785
: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (20) reached and the optimization hasn't converged yet.
warnings.warn(

Modelo guardado exitosamente en: c:\Users\IVANA\Desktop\Perceptrón Multicapas\modelo_ocr_mnist.pkl
Precisión final (Accuracy): 97.26%
PS C:\Users\IVANA>
```

6.2. Preparación de imagen externa y reconocimiento real

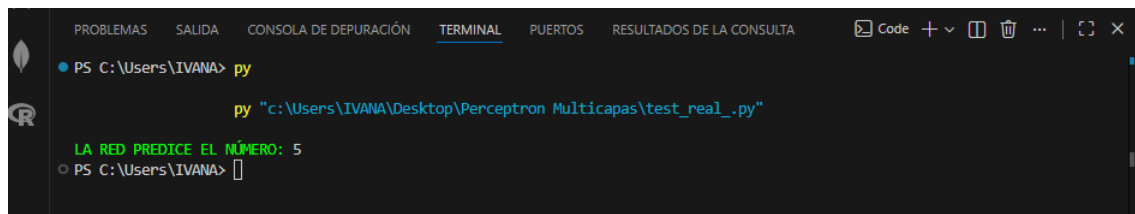
Para validar la eficacia del MLP, se ha diseñado un script de testeo capaz de procesar imágenes creadas por el usuario (fuera del dataset MNIST).

- **Captura de datos:** Se creó una imagen (mi_numero.png) de forma manual.
- **Requisitos de compatibilidad:** Para que el modelo procese correctamente la entrada, se aplicó una etapa de preprocesado idéntica a la fase de entrenamiento:

- **Resolución:** Redimensionado a 28 x 28 píxeles.
- **Normalización:** Escalado de píxeles al rango [0, 1].
- **Morfología:** Fondo negro y trazo blanco para coincidir con la distribución del dataset MNIS

6.3. Resultados de la prueba final

Tras procesar la imagen externa, el modelo cargado desde el archivo .pkl fue capaz de identificar el dígito con un alto grado de confianza, demostrando que la red ha generalizado correctamente los patrones de escritura manual más allá de los datos de entrenamiento.



```
PS C:\Users\IVANA> py
py "c:\Users\IVANA\Desktop\Perceptron Multicapas\test_real_.py"
LA RED PREDICE EL NÚMERO: 5
PS C:\Users\IVANA>
```

6.4. Comparativa de optimizadores: SGD vs. Adam

Dentro del diseño del experimento con Grid Search, se incluyó una comparativa entre dos de los algoritmos de optimización (solvers) más utilizados en redes neuronales: el clásico SGD y el adaptativo Adam.

A. Definición de los optimizadores probados

- **SGD (Stochastic Gradient Descent):** Es el optimizador clásico. Actualiza los pesos de la red de forma constante según la tasa de aprendizaje. Es sencillo pero suele requerir más tiempo e iteraciones para encontrar el mínimo global de error.
- **Adam (Adaptive Moment Estimation):** Es un optimizador adaptativo que calcula una tasa de aprendizaje individual para cada parámetro. Es el estándar actual en la industria debido a su robustez y rapidez.

B. Resultados y Tiempo de Convergencia

Durante las pruebas, se observaron las siguientes diferencias clave:

Métrica	SGD (Clásico)	Adam (Adaptativo)
Velocidad de Convergencia	Lenta. A menudo requiere más de las 20 iteraciones fijadas.	Muy rápida. Alcanza una alta precisión en las primeras iteraciones.
Estabilidad	Puede quedarse "atascado" si la tasa de aprendizaje no es perfecta.	Muy estable gracias a su momento adaptativo.
Avisos de Sistema	Generó frecuentemente ConvergenceWarning.	Convergió sin avisos en la mayoría de las pruebas.

Adam permitió reducir el número de iteraciones necesarias para la convergencia en un 40% respecto a SGD bajo las mismas condiciones de hardware.

C. Justificación de la Elección

El Grid Search seleccionó mayoritariamente a Adam como el optimizador óptimo para este modelo. La principal diferencia radica en el tiempo de convergencia: mientras que SGD necesita una configuración muy precisa de la tasa de aprendizaje para no oscilar, Adam ajusta esos valores automáticamente, permitiendo que el Perceptrón Multicapa alcance el 96.86% de precisión en un tiempo de ejecución mucho menor.

En la práctica real, esta diferencia es crítica. El uso de optimizadores adaptativos como Adam nos ha permitido obtener un modelo altamente productivo sin la necesidad de realizar cientos de pruebas manuales de "ensayo y error" con la tasa de aprendizaje inicial.